

CSD430: Server Side Development

Module 10:

Assignment 2: Creating Custom Tags

Isaac Ellingson

8/2/2026

Creating Custom Tags

What Custom Tags Do

In JSP, custom tags work very similarly to web components from Vue, React, etc. Web components in these frameworks are reusable serverside components that can be included multiple times if necessary in a web page. In modern HTML5, "real" w3c web components can be included clientside by the browser, including sensible fallback behavior for non-webcomponent-capable browsers. In this paper I will primarily discuss JSP in relation to "classic" web components from other serverside frameworks, rather than the modern built-in client functionality.

Advantages of Custom Tags

What I would call components, but JSP calls custom tags, are one way to improve modularity. Let's say you've figured out some HTML and scriptlet code to elegantly display a user profile. Now let's say you've got five user profiles on one page describing your writing staff. Then another page for your editing staff, and another page for your C-suite.

Rather than pasting the same code in multiple times, or calling out to an external method in a bean from a jsp scriptlet tag, wouldn't it be nice to say something like

```
<news:staff person="jwattson" />
```

for each person on the team?

It would certainly be a lot more readable.

Disadvantages

JSP custom tags require a great deal of configuration - you need to create a "TagLib Directive" or .tld file, and you need to create a custom handler class for every single new tag you intend to support. Finally, including the taglib directive with a header in every JSP where it's used. It feels like much of this could be streamlined or automated, but it isn't.

Requirements for a Custom Tag

As mentioned above, the most important pieces are the taglib directive and the handler class. Drilling down into the specifics, a TLD needs a tag declaration with the name, handler class, and optional body content.

This file also requires some boilerplate which is mostly useful for automated verification of the file - XML is often like this, encoding verifier information in the file itself even if the information is redundant across many files of the same type.

```
<?xml version="1.0" encoding="UTF-8"?>
<taglib xmlns="http://www.oracle.com/technetwork/java/index.html"
  xmlns:xsi="https://www.w3.org/2001/XMLSchema-instance"
  version="2.0"
  xsi:schemaLocation="http://www.oracle.com/webfolder/technetwork/jsc/xml/ns/j2ee/web-
    jsptaglibrary_2_0.xsd">

  <tlib-version>2.0</tlib-version>
  <jsp-version>2.0</jsp-version>
  <tag>
    <name>example</name>
    <tag-class>example.Example</tag-class>
    <body-content>empty</body-content>
  </tag>
</taglib>
```

In my example we use a body-content of "empty". That means we can't specify a body for the tag (like "<example>A body</example>"). "scriptless", the default, is available which allows a body of html and custom tags, this is often useful if you're "surrounding" the body with something, like a visual card, an H1, or similar. "tagdependent" means some processing of the raw body text is needed, and is the most complicated of the bunch to use.

"name" here is the name you'll use for the tag, and does not include the prefix - the prefix is set by the page doing the include.

The handler class itself must extend either Tag (to use the old JSP custom tag API) or

```
package example;

import java.io.IOException;
import javax.servlet.jsp.JspException;
import javax.servlet.jsp.JspWriter;
import javax.servlet.jsp.tagext.SimpleTagSupport;

public class Example extends SimpleTagSupport {
    @Override
    public void doTag() throws JspException, IOException {
        JspWriter out = getJspContext().getOut();
        out.println("This is an example!");
    }
}
```

SimpleTag (recommended). A SimpleTagSupport concrete class is provided which does nothing, and provides a neutral superclass so you don't have to override everything. You *would* override doTag to perform any necessary processing and output for your tag.

Additional rules exist for a kind of additional dependency injection to share objects within hierarchies of cooperating tags, but they are well outside the scope of this introductory overview.

My Thoughts

Any modularity you can squeeze into a system is good. The fewer pieces of a system we need to understand to read a given part of its source, the better. JSP tags help "banish" complexity from pages that really should be "about" content and layout, not code.

Also, moving from imperative systems to declarative language is good, when you can do it. As Einstein (or perhaps Roger Sessions) said, make things as simple as you can, but no simpler. Mojang has a lot to overdo it - json data that, on its face is declarative, but which is used to describe imperative functions, and that simply does not work. So use declarative language with wisdom and care. Here I believe it could make a huge readability difference, since often we are describing repeated page elements which logically *should* be HTML tags.

I've avoided talking about "native" web components on the client, but in almost all respects, native web components are vastly superior. If it's just about how the elements are presented on the client, let's put that stuff *on* the client and ask the browser to render it. The main place where I can see a JSP tag shining serverside, is when the custom tag then interacts with a service bean to grab additional data that's only available on the server, and organize that data for the browser to display.

References

Body-content attribute (the java ee 5 tutorial). (2026a). Oracle.Com.

<https://docs.oracle.com/cd/E19316-01/819-3669/bnamg/index.html>

GeeksforGeeks. (2022a). Custom tags in JSP. In *GeeksforGeeks*.

<https://www.geeksforgeeks.org/java/custom-tags-in-jsp/>

GeeksforGeeks. (2022b, December 28). *Custom tags with body in JSP*. GeeksforGeeks.

<https://www.geeksforgeeks.org/java/custom-tags-with-body-in-jsp/>

Processing body of custom tag in JSP 2.0. (2026b). Srikanthtechnologies.Com.

https://www.srikanthtechnologies.com/blog/java/process_ct_body.aspx

SimpleTagSupport (Java(TM) EE 7 specification APIs). (2015, June). Oracle.Com.

<https://docs.oracle.com/javaee/7/api/javax/servlet/jsp/tagext/SimpleTagSupport.html#doTag-->

oTag--